# Internal Study Guide

## Evaluating Large Language Models for Clinical Decision-Making Using AgentClinic

***Extreme-Detail Viva Preparation Document***

Dikshant Sharma
IIT Kharagpur · Dept. of Artificial Intelligence
Roll No: 21IM3AI29

April 2026

*For internal study ONLY — not for submission.*
*Covers every concept with academic definitions, simple analogies, worked examples, and potential viva questions.*

# Contents


**1 Chapter 1: Introduction** **3**
- 1.1 Main Objective of the Project . . . 3
  - 1.1.1 Central Research Question . . . 3
  - 1.1.2 How We Achieved It — Three Phases . . . 3
  - 1.1.3 The Answer to the Research Question . . . 5
- 1.2 Why Static Medical QA Benchmarks Are Insufficient . . . 5
  - 1.2.1 One-Shot Factual Recall vs. Interactive Multi-Turn Reasoning . . . 6
- 1.3 What is Lexical Leakage? . . . 6
- 1.4 Problem Statement: Four Key Research Gaps . . . 7
- 1.5 Contributions (Five Key Contributions) . . . 7
- 1.6 Potential Viva Questions — Chapter 1 . . . 9

**2 Chapter 2: Literature Review** **10**
- 2.1 Medical Question Answering Benchmarks . . . 10
- 2.2 Dialogue-Based Simulations: AMIE and Agent Hospital . . . 10
  - 2.2.1 AMIE (Articulate Medical Intelligence Explorer) . . . 10
  - 2.2.2 Agent Hospital . . . 11
- 2.3 Cognitive Biases in Clinical AI . . . 11
- 2.4 Open-Source LLMs: Architectures . . . 12
- 2.5 Quantisation Techniques . . . 13
  - 2.5.1 Why Quantisation is Needed . . . 13
  - 2.5.2 NF4 (NormalFloat4) Quantisation . . . 13
  - 2.5.3 Double Quantisation . . . 14
  - 2.5.4 QLoRA . . . 14
- 2.6 Summary of Five Research Gaps . . . 14
- 2.7 Potential Viva Questions — Chapter 2 . . . 15

**3 Chapter 3: Dataset — AgentClinic-MedQA** **16**
- 3.1 What is an OSCE? . . . 16
- 3.2 The JSON Schema . . . 16
- 3.3 Knowledge Partitioning . . . 17
- 3.4 Semantic Evaluation by the Moderator Agent . . . 18
- 3.5 Dataset Statistics . . . 18
- 3.6 Potential Viva Questions — Chapter 3 . . . 19

**4 Chapter 4: Initial Framework (Phase 1)** **20**
- 4.1 Architecture: Ad-Hoc While-Loop Orchestrator . . . 20
- 4.2 Models Tested and Results . . . 20
- 4.3 Catastrophic Forgetting . . . 21
- 4.4 Bias Impact Analysis (Phase 1) . . . 22
- 4.5 Three Critical Architectural Flaws Identified . . . 22
- 4.6 Potential Viva Questions — Chapter 4 . . . 23

**5 Chapter 5: Enhanced Methodology — AgentClinic v2** **24**
- 5.1 LangGraph Directed Cyclic Graph (DCG) . . . 24
  - 5.1.1 What is a State Machine? . . . 24

5.1.2 What is LangGraph? . . . 24
5.1.3 Why a Cyclic Graph (DCG) and Not a DAG? . . . 25
5.2 The Latent Metacognitive Reflection Node . . . 25
5.3 ClinicalState TypedDict . . . 26
5.4 Deterministic Routing via `route_doctor()` . . . 27
5.5 Heterogeneous Multi-Engine Architecture . . . 27
5.6 4-Bit NF4 Quantisation with Double Quantisation . . . 28
5.7 Flash Attention 2 . . . 28
5.8 Cognitive Bias Injection Manifold . . . 29
5.8.1 Recency Bias ($k = 5$ Case Memory Queue) . . . 29
5.8.2 Confirmation Bias (Strong Prior Hypothesis Injection) . . . 29
5.9 Task-Specific Dynamic LoRA Adapter Routing . . . 29
5.9.1 The Problem: Why Can't We Just Fine-Tune the Whole Model? . . . 30
5.9.2 What is LoRA? — Deep Dive . . . 31
5.9.3 The Scaling Factor $\alpha$ . . . 32
5.9.4 Which Layers Are Modified? (Target Modules) . . . 33
5.9.5 The Two Adapters: Reasoning and Dialogue . . . 34
5.9.6 How Adapter Routing Works in One Turn . . . 35
5.9.7 Training the Adapters: QLoRA Pipeline . . . 36
5.9.8 Why Dynamic LoRA Resolves the Trade-off . . . 37
5.10 Theoretical Underpinnings . . . 37
5.10.1 Decoupling Reasoning from Generation (Latent CoT) . . . 37
5.10.2 Embedding Manifold Overlap and Lexical Leakage . . . 37
5.10.3 Information-Theoretic Basis of NF4 . . . 38
5.11 Potential Viva Questions — Chapter 5 . . . 39

**6 Chapter 6: Experiments and Results 40**
6.1 Experimental Design . . . 40
6.2 Full Results Table . . . 40
6.3 Metric Definitions with Worked Examples . . . 41
6.3.1 Diagnostic Stability (DS) . . . 41
6.3.2 Test Rationality (TR) . . . 42
6.3.3 Information Efficiency (IE) . . . 42
6.4 Five Key Findings Explained . . . 43
6.4.1 Finding 1: E1 Accuracy (45.6%) Increased from Phase 1 (44%) . . . 43
6.4.2 Finding 2: Why E3 > E2 (Instruction Alignment vs. Domain Fine-Tuning) . . . 43
6.4.3 Finding 3: E4 Recency Bias — "Hypothesis Thrashing" . . . 43
6.4.4 Finding 4: E5 Confirmation Bias — "Premature Diagnostic Closure" 43
6.4.5 Finding 5: E6 Dynamic LoRA — Highest Accuracy . . . 44
6.5 Potential Viva Questions — Chapter 6 . . . 45

**7 Chapter 7: Discussion 46**
7.1 Structured Metacognition Eclipses Lexical Leakage . . . 46
7.2 Domain Specialisation vs. Instruction Alignment . . . 47
7.3 Dynamic LoRA Resolves the Trade-off . . . 48
7.4 Safety Threat of Cognitive Biases . . . 48
7.5 Implications for Clinical AI Evaluation Standards . . . 49
7.6 Limitations . . . 49

7.7 Potential Viva Questions — Chapter 7 . . . . . 51

**8 Chapter 8: Conclusion and Future Work 52**
8.1 Summary of Findings (5 Bullet Points) . . . . . 52
8.2 Future Work . . . . . 53
8.2.1 Process Reward Modelling via Direct Preference Optimisation (DPO) 53
8.2.2 RL from AI Feedback via Self-Play . . . . . 54
8.2.3 Extending LoRA to Bias Resilience . . . . . 55
8.2.4 Multimodal Integration . . . . . 55
8.3 Potential Viva Questions — Chapter 8 . . . . . 57

**9 Appendix A: Complete Code Workflow 58**
9.1 High-Level Architecture Overview . . . . . 58
9.2 Step-by-Step Execution Flow . . . . . 58
9.2.1 Step 1: CLI Invocation and Argument Parsing (Section 9) . . . . . 58
9.2.2 Step 2: Heterogeneous Engine Loading (Section 3 via Section 8) . . 59
9.2.3 Step 3: Dataset Loading (Section 5) . . . . . 60
9.2.4 Step 4: Scenario Loop (Section 8, repeated 107 times) . . . . . 60
9.2.5 Step 5: LangGraph Construction (Section 7 + Section 8) . . . . . 61
9.2.6 Step 6: Graph Execution (One Full Consultation) . . . . . 62
9.2.7 Step 7: Evaluation (Moderator + Metrics) . . . . . 63
9.2.8 Step 8: Results Persistence . . . . . 64
9.2.9 Step 9: Final Summary . . . . . 64
9.3 Complete Data Flow Diagram . . . . . 64
9.4 Potential Viva Questions — Code Workflow . . . . . 67

# 1 Chapter 1: Introduction

## 1.1 Main Objective of the Project

**Project Objective**

**Title:** Design and Evaluation of Multi-Agent LLM Systems for Interactive Clinical Diagnosis

**Objective:** To design, implement, and rigorously evaluate a multi-agent simulation framework that tests open-source LLMs not on *what* they diagnose, but on *how* they reason—capturing the full cognitive trajectory of a clinical consultation rather than just the final answer.

### 1.1.1 Central Research Question

**The Core Question**

*Can open-source LLMs, when placed inside a structured multi-agent simulation that mirrors real clinical workflows, demonstrate safe and rational diagnostic reasoning—and can we measure that reasoning beyond simple accuracy?*

### 1.1.2 How We Achieved It — Three Phases

**Phase 1: Baseline Benchmarking (Initial Framework):**

- Built a multi-agent simulation with Doctor, Patient, and Measurement agents using an ad-hoc `while`-loop
- Benchmarked 4 open-source LLMs (Mistral-7B, BioGPT, JSL-Med, Apollo)
- Discovered *Lexical Leakage* and *catastrophic forgetting* in medically fine-tuned models
- **Key finding:** Generalist Mistral-7B (44%) outperformed all specialist models

**Phase 2: Architecture Redesign (AgentClinic v2):**

- Replaced the ad-hoc loop with a **LangGraph Directed Cyclic Graph** state machine

- Implemented **heterogeneous multi-engine isolation** (NF4 quantisation) to eliminate Lexical Leakage
- Introduced the **latent metacognitive reflection node** to make internal reasoning observable
- Formalised three process metrics: Diagnostic Stability, Test Rationality, Information Efficiency
- **Key finding:** Structured metacognition alone improved accuracy from 44.0% to 45.6%, compensating for Lexical Leakage removal

**Phase 3: Advanced Experimentation (E1–E6):**

- Ran 6 experiments across 107 MedQA scenarios with bias injection and LoRA routing
- Showed that instruction alignment (E3: 53.5%) matters more than domain fine-tuning (E2: 50.2%) for interactive tasks
- Demonstrated that cognitive biases are a critical safety threat (E4: 38.1%, E5: 31.4%)
- Resolved the specialisation–alignment trade-off with Dynamic LoRA adapters (E6: 54.7%)
- **Key finding:** Process metrics reveal dangerous failure modes that accuracy alone completely masks

### 1.1.3 The Answer to the Research Question

**Summary Answer**

**Yes, but with critical caveats.** Open-source LLMs can perform interactive clinical reasoning when given proper architectural scaffolding (LangGraph DCG + metacognitive reflection). However:

- The best accuracy achieved was 54.7% (E6 with Dynamic LoRA)—roughly 1 in 2 diagnoses are wrong
- Cognitive biases degrade performance catastrophically (down to 31.4%) and can be *invisible* to accuracy-only evaluation
- Process metrics (DS, TR, IE) are essential for safety evaluation—a model can appear "confidently consistent" ($DS = 0.88$) while being systematically wrong

**Conclusion:** Clinical LLMs are not ready for deployment, but the evaluation methodology developed here provides the tools to measure and improve their readiness.

## 1.2 Why Static Medical QA Benchmarks Are Insufficient

**Academic Definition**

Static medical QA benchmarks—such as MedQA, USMLE Step examinations, PubMedQA, and MedMCQA—evaluate a model's ability to select the correct answer from a fully-specified clinical vignette presented in a single prompt. All patient history, vital signs, and investigation results are provided upfront. These benchmarks measure **declarative medical knowledge retrieval**, not **procedural clinical reasoning**.

**Simple Analogy**

Imagine an exam where the question already tells you everything: "A 55-year-old man has chest pain, elevated troponin, and ST-elevation on ECG. What is the diagnosis?" That's MedQA—it's an open-book exam where the answer is almost in the question. Now imagine a *real* doctor who sees a patient walk in and say "My chest hurts." The doctor must decide: What questions do I ask? What tests do I order? Should I get an ECG first or blood work? *That* is what AgentClinic tests—the thinking process, not just the final answer.

**Why It Matters for This Thesis**

This thesis argues that passing MedQA $\neq$ being a competent clinical AI. The entire motivation for building AgentClinic is to create an evaluation that tests the *journey*, not just the *destination*.

### 1.2.1 One-Shot Factual Recall vs. Interactive Multi-Turn Reasoning

- **One-shot factual recall:** All information is given at once. The model selects from pre-defined choices. No follow-up, no uncertainty, no evidence acquisition.
- **Interactive multi-turn reasoning:** Information is *hidden*. The model must actively decide what to ask, what to test, how to update hypotheses, and when to stop. This mirrors real clinical practice where diagnosis unfolds over 10–30 minutes of iterative doctor–patient interaction.

## 1.3 What is Lexical Leakage?

**Academic Definition**

**Lexical Leakage** occurs in homogeneous multi-agent LLM simulations where the same model weights ($\theta_{\text{shared}}$) are used for both the Doctor and Patient agents. Because both agents project text into identical high-dimensional embedding manifolds, the Doctor can perform **zero-shot knowledge transfer** by recognising the mathematical proximity of the Patient's generated token embeddings within this shared space.

Formally: when $\text{Doctor}(\theta) = \text{Patient}(\theta)$, the Patient's symptom embeddings $\mathbf{h}_{\text{patient}} = f_\theta(\text{symptoms})$ and the Doctor's disease representations $\mathbf{h}_{\text{doctor}} = f_\theta(\text{disease})$ occupy **overlapping regions** of the same embedding manifold. The Doctor may infer ground-truth diagnostic information not through genuine clinical reasoning, but through implicit geometric similarity detection in the latent space.

**Simple Analogy**

Imagine an exam where the student and the answer-key-maker share the same brain. The student doesn't "cheat" consciously—but because they share the same neural wiring, the student subconsciously recognises patterns from the answer key. That's Lexical Leakage: the Doctor doesn't explicitly see the Patient's ground truth, but the shared mathematical structure of their representations creates an unfair back-channel of information.

**Why It Matters for This Thesis**

In the initial framework, a single Mistral-7B was used for *both* Doctor and Patient. The 44% accuracy achieved might have been partially inflated by Lexical Leakage. This motivated the heterogeneous multi-engine architecture in AgentClinic v2.

## 1.4 Problem Statement: Four Key Research Gaps

This thesis addresses four critical gaps:

**Gap 1: No interactive open-source frameworks** — Existing systems rely on proprietary APIs (GPT-4) or lack the modularity for controlled experimentation.

**Gap 2: Lexical Leakage vulnerability** — Homogeneous multi-agent systems allow the Doctor to implicitly access ground truth through shared embedding manifolds.

**Gap 3: No process evaluation** — Only terminal diagnostic accuracy is measured; the cognitive trajectory, stability, and rationality of reasoning are ignored.

**Gap 4: No controlled bias modelling** — No systematic mechanism exists to inject and measure the impact of cognitive biases (anchoring, confirmation, recency) on LLM clinical reasoning.

## 1.5 Contributions (Five Key Contributions)

1. **Baseline open-source benchmarking:** First systematic evaluation of 4 open-source LLMs (Mistral-7B, BioGPT, JSL-Med, Apollo) in interactive clinical simulation, revealing Lexical Leakage and catastrophic forgetting.

2. **Lexical Leakage mitigation:** Heterogeneous multi-engine architecture using NF4 double quantisation to run 3+ distinct models on a single GPU.
3. **LangGraph DCG + Metacognitive Reflection:** Deterministic state machine with a latent reflection node that makes internal reasoning observable.
4. **Three novel process metrics:** Diagnostic Stability (DS), Test Rationality (TR), Information Efficiency (IE).
5. **Dynamic LoRA adapter routing:** Task-specific adapters for reasoning vs. dialogue, achieving the highest accuracy (54.7%) by resolving the specialisation–alignment trade-off.

## 1.6 Potential Viva Questions — Chapter 1

**Likely Viva Questions**

1. **Q:** Why can't we just use MedQA scores to evaluate clinical AI?

   **A:** MedQA provides all information upfront. Real diagnosis requires iterative evidence acquisition, hypothesis refinement, and uncertainty management—skills MedQA never tests.

2. **Q:** Can you explain Lexical Leakage in mathematical terms?

   **A:** When Doctor and Patient share weights $\theta$, their embedding functions $f_\theta$ produce vectors in the same manifold. The Doctor detects geometric proximity between symptom embeddings and disease embeddings, gaining implicit access to ground truth.

3. **Q:** Why is Lexical Leakage dangerous rather than helpful?

   **A:** It inflates accuracy artificially, making the model appear more capable than it actually is. In deployment, the Doctor would interact with real patients (not LLM-generated ones), so this back-channel would not exist.

4. **Q:** What is your most important contribution?

   **A:** The latent metacognitive reflection node, which forces the LLM to "think before it speaks," enabling trajectory-aware evaluation and demonstrating that structured metacognition eclipses Lexical Leakage.

5. **Q:** How does your work differ from AMIE or Agent Hospital?

   **A:** AMIE is closed-source and proprietary. Agent Hospital uses homogeneous agents (vulnerable to Lexical Leakage). Our system is open-source, heterogeneous, deterministic, and provides process-level metrics.

# 2 Chapter 2: Literature Review

## 2.1 Medical Question Answering Benchmarks

**Academic Definition**

**MedQA** is a large-scale open-domain QA dataset sourced from the US Medical Licensing Examination (USMLE). Models receive a complete patient vignette and select from multiple-choice answers. Similar benchmarks include **PubMedQA** (biomedical literature comprehension), **MedMCQA** (Indian medical entrance exams), and **MMLU Clinical Topics** (multi-task language understanding).
GPT-4 has achieved >90% accuracy on USMLE, surpassing expert human scores. However, these benchmarks test only **knowledge retrieval from a pre-specified context**—they never require the model to decide what information to seek.

**Simple Analogy**

MedQA is like a "fill-in-the-blank" test where all the clues are already on the page. The student just needs to recognise the pattern. AgentClinic is like a practical exam where you walk into a room, meet a patient, and must figure out what's wrong from scratch—asking the right questions, ordering the right tests.

**What they test:** Factual recall, clinical guideline application, pattern recognition.
**What they miss:** Information acquisition strategy, hypothesis evolution, multi-turn dialogue management, test ordering rationality, clinical uncertainty handling.

## 2.2 Dialogue-Based Simulations: AMIE and Agent Hospital

### 2.2.1 AMIE (Articulate Medical Intelligence Explorer)

**Academic Definition**

AMIE is Google's conversational diagnostic AI system. It demonstrated improved diagnostic accuracy in simulated multi-turn conversations and was evaluated by both clinicians and patients against real physicians. **Limitations:** AMIE is closed-source, relies entirely on proprietary models, and lacks the modularity required for scientific replication or controlled bias injection.

### 2.2.2 Agent Hospital

**Academic Definition**

Agent Hospital provides an evolvable hospital simulacrum using LLM agents. It simulates patient triage, diagnosis, and treatment workflows. **Critical limitation:** It uses **homogeneous agent designs**—the same model for all roles—which this thesis identifies as vulnerable to *Lexical Leakage*. Furthermore, it functions as an end-to-end benchmark rather than an extensible research tool.

**Why It Matters for This Thesis**

Neither AMIE nor Agent Hospital provides: (a) open-source access, (b) heterogeneous agent isolation, (c) controlled bias injection, or (d) process-oriented metrics. AgentClinic v2 addresses all four.

## 2.3 Cognitive Biases in Clinical AI

**Academic Definition**

**Cognitive biases** are systematic deviations from normative reasoning that affect human (and LLM) diagnostic decisions. Three are studied in this thesis:

1. **Anchoring Bias:** Over-reliance on the first piece of information encountered.
   *Medical example:* A doctor sees "chest pain" and immediately anchors on "heart attack," ignoring later evidence suggesting costochondritis (rib inflammation).

2. **Confirmation Bias:** Seeking evidence that confirms an existing hypothesis while ignoring contradictory evidence.
   *Medical example:* A doctor suspects pneumonia, orders a chest X-ray (confirming), but ignores the patient's travel history suggesting malaria.

3. **Recency Bias:** Disproportionate weight given to the most recently encountered cases.
   *Medical example:* After seeing 5 consecutive cardiac patients, a doctor diagnoses the 6th patient (who has acid reflux) with a cardiac condition.

**Simple Analogy**

**Anchoring:** Like a GPS that locks onto the first satellite it finds and refuses to recalculate.

**Confirmation:** Like only reading news that agrees with your opinion.

**Recency:** Like a sports commentator who predicts tomorrow's match based only on yesterday's result.

## 2.4 Open-Source LLMs: Architectures

**Academic Definition**

All models used in this thesis are **decoder-only transformers**—autoregressive models that predict the next token conditioned on all previous tokens: $P(w_t \mid w_1, \ldots, w_{t-1})$.

| Model | Parameters | Architecture | Training Focus |
|---|---|---|---|
| Mistral-7B-Instruct-v0.2 | 7.24B | Decoder-only, GQA, SWA | General instruction-tu |
| Llama-3.1-8B-Instruct | 8.03B | Decoder-only, GQA | Instruction alignment |
| Mixtral-8x7B | 46.7B (12.9B active) | Mixture-of-Experts | Sparse expert routing |
| JSL-MedMistral-7B | 7.24B | Mistral base + medical SFT | Clinical/biomedical |
| BioGPT (1.5B) | 1.5B | Decoder-only | PubMed abstracts |

**Key architectural features:**

- **GQA (Grouped-Query Attention):** Shares key-value heads across query groups, reducing KV-cache memory by $\sim 4\times$.
- **SWA (Sliding Window Attention):** Mistral uses a 4096-token sliding window, enabling efficient long-context processing.
- **MoE (Mixture of Experts):** Mixtral routes each token to 2 of 8 expert FFN layers, using only 12.9B of 46.7B parameters per forward pass.

## 2.5 Quantisation Techniques

### 2.5.1 Why Quantisation is Needed

A 7B-parameter model in FP16 (16-bit floating point) requires:

$$7 \times 10^9 \text{ params} \times 2 \text{ bytes/param} = 14 \text{ GB VRAM}$$

Running 3 heterogeneous models simultaneously would require ∼42 GB in FP16 *before* accounting for KV-cache and activation memory—infeasible on most GPUs.

### 2.5.2 NF4 (NormalFloat4) Quantisation

**Academic Definition**

**NormalFloat4 (NF4)** is an information-theoretically optimal 4-bit data type for normally distributed data. Pre-trained neural network weights typically follow a zero-mean Gaussian distribution $\mathcal{N}(0, \sigma^2)$. Standard uniform quantisation (INT4/INT8) wastes bit capacity on sparse outlier regions. NF4 creates **non-linearly spaced quantisation bins** that place more bins in the high-density central region where the majority of weights reside, and fewer bins in the tails.

**Worked Example: Memory Savings (FP32 → NF4)**

**FP32 (32-bit):**

$$7\text{B} \times 4 \text{ bytes} = 28 \text{ GB}$$

**FP16 (16-bit):**

$$7\text{B} \times 2 \text{ bytes} = 14 \text{ GB}$$

**NF4 (4-bit):**

$$7\text{B} \times 0.5 \text{ bytes} = 3.5 \text{ GB}$$

Plus quantisation constants overhead (∼0.3 GB) ⇒ **≈3.8 GB total**.

**Reduction:** $\frac{28-3.8}{28} \times 100 = 86.4\%$ memory savings from FP32.

### 2.5.3 Double Quantisation

**Academic Definition**

Standard block-wise quantisation stores a FP32 scaling constant per block of 64 weights, adding $\frac{32}{64} = 0.5$ bits per parameter. **Double quantisation** applies a second round of quantisation *to the quantisation constants themselves*, compressing them from FP32 to FP8. This recovers ≈0.37 bits per parameter:

$$\text{Savings} = 0.5 - \frac{8}{64} - \frac{32}{64 \times 256} = 0.37 \text{ bits/param}$$

**Simple Analogy**

Imagine you compress a photo. The compression algorithm uses a "codebook" to remember how to decompress. Double quantisation is like compressing the codebook itself—saving space on the thing that saves space.

### 2.5.4 QLoRA

**Academic Definition**

**QLoRA (Quantised LoRA)** combines NF4 quantisation of base model weights with Low-Rank Adaptation (LoRA) adapters trained in FP16/BF16. The base model is frozen in 4-bit; only the small LoRA matrices ($\Delta W = BA$) are trainable in full precision. This allows fine-tuning of a 7B model on a single consumer GPU (∼12 GB VRAM).

## 2.6 Summary of Five Research Gaps

1. Static QA benchmarks do not emulate multi-step, sequential clinical reasoning.
2. Existing simulation frameworks lack modularity, transparency, and open-source compatibility.
3. Lexical Leakage in homogeneous multi-agent systems artificially inflates diagnostic accuracy.
4. Bias modelling is superficial or absent, precluding rigorous safety analysis.
5. Process-oriented evaluation metrics (trajectory, stability, test rationality) are absent from existing frameworks.

## 2.7 Potential Viva Questions — Chapter 2

**Likely Viva Questions**

1. **Q:** What is the key difference between MedQA and AgentClinic?

   **A:** MedQA provides all information upfront for one-shot recall; AgentClinic requires iterative, multi-turn evidence acquisition and hypothesis refinement.

2. **Q:** Why are AMIE's results not directly comparable to yours?

   **A:** AMIE uses proprietary, closed-source models and does not provide open-source access for replication. Our work uses exclusively open-source models.

3. **Q:** Explain NF4 quantisation in one sentence.

   **A:** NF4 maps neural network weights—which are approximately Gaussian—into 4-bit bins that are optimally spaced for that distribution, preserving statistical fidelity while reducing memory by $\sim$86%.

4. **Q:** What is double quantisation and how much does it save?

   **A:** It quantises the FP32 scaling constants used in block-wise quantisation down to FP8, recovering $\approx$0.37 bits per parameter.

5. **Q:** Why did you choose Mistral-7B as a baseline over GPT-4?

   **A:** Our thesis focuses on open-source, locally deployable models. GPT-4 results exist in the literature but are not reproducible, not transparent, and impractical for controlled multi-agent simulations requiring heterogeneous engine isolation.

6. **Q:** What is a decoder-only transformer?

   **A:** A model that generates text left-to-right, predicting each token based on all previous tokens: $P(w_t|w_{<t})$. Unlike encoder-decoder models (T5), it has no separate encoding stage.

7. **Q:** How does confirmation bias differ from anchoring bias?

   **A:** Anchoring is over-reliance on the *first* piece of information. Confirmation is selectively seeking evidence that *supports* an existing hypothesis. Anchoring creates the hypothesis; confirmation bias maintains it.

# 3 Chapter 3: Dataset — AgentClinic-MedQA

## 3.1 What is an OSCE?

**Academic Definition**

An **Objective Structured Clinical Examination (OSCE)** is a standardised medical assessment format used globally in medical education. Students rotate through "stations," each presenting a simulated patient scenario. At each station, the student must take a history, perform a physical examination, order tests, and formulate a diagnosis—all within a time limit. OSCEs test procedural competence, not just factual recall.

**Simple Analogy**

An OSCE is like a driving test: you don't just answer questions about road rules (that's the written test, analogous to MedQA). You actually sit in the car, drive on real roads, parallel park, and handle unexpected situations. The examiner watches your *process*, not just whether you reach the destination.

**Why It Matters for This Thesis**

AgentClinic converts static MedQA vignettes into OSCE-style interactive simulations. Each scenario becomes a "station" where the LLM Doctor must navigate a multi-turn conversation, request specific tests, and declare a diagnosis—mirroring the OSCE format.

## 3.2 The JSON Schema

Each scenario in the AgentClinic-MedQA dataset is parsed into a structured JSON format with five distinct modules:

1. `Objective_for_Doctor` — A string describing the consultation objective.
   *Example:* "Evaluate and diagnose the patient presenting with chest pain and shortness of breath."
   **Visibility:** Doctor Agent only.

2. `Patient_Actor` — Contains:
   - Demographics (age, gender, social background)

- History of Present Illness (HPI)
- Primary and secondary symptoms
- Past Medical and Social History

**Visibility:** Patient Agent only.

3. `Test_Results` — Laboratory results (CBC, BMP, cardiac enzymes), imaging summaries (X-ray, CT, MRI), and specialised tests (ECG, spirometry).
   **Visibility:** Measurement Agent only. Disclosed to Doctor *only* upon explicit, well-formed test request.
4. `Physical_Examination_Findings` — Inspection, auscultation, palpation, and percussion findings (e.g., "bilateral crackles on lung auscultation").
   **Visibility:** Measurement Agent only.
5. `Correct_Diagnosis` — The verified ground-truth disease label.
   **Visibility:** Moderator Agent only. No active agent ever sees this.

## 3.3 Knowledge Partitioning

**Academic Definition**

**Knowledge partitioning** is the strict information-theoretic separation of case data across agent roles. Each agent receives only its designated "slice" of the scenario. The Doctor sees only the consultation objective; the Patient holds the symptom narrative; the Measurement Agent holds test results; and the Moderator holds the ground truth. No agent can access another's partition.

**Simple Analogy**

Think of a relay race where each runner carries a different piece of a treasure map. Runner 1 (Doctor) has the compass directions. Runner 2 (Patient) has the landmark descriptions. Runner 3 (Measurement) has the distances. Runner 4 (Moderator) has the answer. No single runner can find the treasure alone—they must communicate and share information step-by-step.

**Why It Matters for This Thesis**

Knowledge partitioning is what makes AgentClinic a genuine test of clinical reasoning. If the Doctor could see the Patient's full symptom profile or the test results upfront, it would reduce to a static QA task. The partitioning forces the Doctor to *earn* information through targeted questioning—exactly as a real physician must.

## 3.4 Semantic Evaluation by the Moderator Agent

**Academic Definition**

The Moderator Agent evaluates diagnostic correctness using **LLM-mediated semantic equivalence comparison** rather than exact string matching. Given the Doctor's predicted diagnosis $d_{\text{pred}}$ and the ground truth $d_{\text{true}}$, the Moderator outputs a binary judgement ("Yes" / "No") after assessing whether the two are semantically equivalent.

**Why not exact string matching?**

Consider the following clinically equivalent pairs:

| Doctor's Prediction | Ground Truth |
|---|---|
| "Heart Attack" | "Acute Myocardial Infarction" |
| "Broken Arm" | "Distal Radius Fracture" |
| "Blood Clot in Lung" | "Pulmonary Embolism" |

Exact string matching would mark all of these as *incorrect*, despite being perfectly valid diagnoses. The LLM-mediated Moderator understands medical synonymy.

**Simple Analogy**

It's like having a bilingual teacher grade an exam. If a student answers "Herzinfarkt" (German for heart attack) and the answer key says "Myocardial Infarction," a rigid grading machine fails them—but the bilingual teacher knows they mean the same thing.

## 3.5 Dataset Statistics

- **Total scenarios:** 107 high-quality, diverse diagnostic cases
- **Source:** AgentClinic-MedQA subset (derived from USMLE-style vignettes)

- **Organ system coverage:** Cardiovascular, respiratory, gastrointestinal, neurological, musculoskeletal, endocrine, renal, infectious disease
- **Maximum turns per scenario:** 10 multi-turn inference steps

## 3.6 Potential Viva Questions — Chapter 3

**Likely Viva Questions**

1. **Q:** What is an OSCE and how does AgentClinic simulate it?

   **A:** An OSCE is a practical clinical exam with simulated patient "stations." AgentClinic converts each MedQA vignette into an OSCE station where the LLM Doctor must interactively extract information, order tests, and diagnose—replicating the procedural nature of an OSCE.
2. **Q:** Why did you use only 107 scenarios?

   **A:** Each scenario requires multiple LLM inference passes per turn (Doctor, Patient, Measurement, Moderator) across 10 turns. With 4-bit quantised models on a single H100, each full experiment takes substantial compute. 107 scenarios provide sufficient statistical power while remaining computationally feasible.
3. **Q:** Why not use exact string matching for evaluation?

   **A:** Medical terminology has extensive synonymy. "Heart Attack" and "Acute Myocardial Infarction" are identical diagnoses but different strings. LLM-mediated semantic evaluation captures clinical equivalence.
4. **Q:** What does knowledge partitioning prevent?

   **A:** It prevents the Doctor from having access to information it hasn't earned through dialogue. Without partitioning, the evaluation reduces to a static QA task, defeating the purpose of interactive simulation.
5. **Q:** Could the Moderator itself make errors in semantic comparison?

   **A:** Yes—this is a known limitation. However, LLM-mediated semantic comparison has been shown to correlate highly with expert human judgement, and is superior to rigid string matching for clinical terminology.

# 4 Chapter 4: Initial Framework (Phase 1)

## 4.1 Architecture: Ad-Hoc While-Loop Orchestrator

**Academic Definition**

The initial framework employed a Python `while`-loop to sequentially route messages between the Doctor, Patient, and Measurement agents. The loop iterated for a fixed number of turns (typically 10) or until the Doctor emitted a `DIAGNOSIS READY` token. This design was functional but lacked formal state management, deterministic routing, or trajectory recording.

**Simple Analogy**

Imagine a school play where the actors have no script. A stage manager just says "your turn, your turn, your turn" in a loop. Sometimes actors talk out of order, sometimes they freeze, and nobody records what happened. That's the while-loop orchestrator—it works most of the time, but has no formal rules and breaks unpredictably.

## 4.2 Models Tested and Results

Four open-source models were evaluated on 107 AgentClinic-MedQA scenarios:

| Model | Training Data Alignment | Accuracy (%) |
|---|---|---|
| Mistral-7B-Instruct-v0.2 | General instruction-tuning | **44.0** |
| Microsoft BioGPT (1.5B) | Biomedical literature (PubMed) | 35.0 |
| JSL-Med-Sft-Llama-3-8B | Clinical + Biomedical SFT | 32.0 |
| Apollo (7B) | General text corpora | 23.0 |

**The paradox:** The generalist Mistral-7B (44%) *outperformed* all medically fine-tuned models. This demands explanation.

## 4.3 Catastrophic Forgetting

**Academic Definition**

**Catastrophic forgetting** occurs when fine-tuning a pre-trained neural network on a narrow downstream task causes the model to "forget" the broad capabilities it learned during pre-training. In the context of medical SFT (Supervised Fine-Tuning), optimising on static QA pairs (e.g., USMLE-style questions) erodes the model's general-purpose instruction-following, conversational coherence, and multi-step logical reasoning abilities.

**Simple Analogy**

Imagine a chess grandmaster who decides to study *only* the Sicilian Defence opening for 6 months. After that intensive study, they know every Sicilian variation perfectly—but they've forgotten how to play the middlegame and endgame. They lose to average players in positions outside their narrow specialty. That's catastrophic forgetting: narrow expertise destroying broad competence.

**Why It Matters for This Thesis**

BioGPT (35%) and JSL-Med (32%) possess superior biomedical *knowledge* but have lost the conversational *skills* needed to navigate a 10-turn interactive simulation. Mistral-7B's broader instruction-tuning preserved its ability to ask follow-up questions, interpret test results contextually, and maintain coherent dialogue flow. This finding directly motivated the Dynamic LoRA approach in v2 (Chapter 5), which separates domain knowledge from conversational skills.

**Why does this happen mechanically?**

- Medical SFT datasets are typically formatted as single-turn QA pairs: "Question: [vignette]. Answer: [diagnosis]."
- Training on these pairs optimises the model's attention patterns for *immediate* pattern-matching, not for multi-turn hypothesis refinement.
- The gradient updates during SFT overwrite the attention weight distributions that previously supported conversational coherence and sequential reasoning.
- Result: the model can answer "What disease has these symptoms?" but cannot ask "What other symptoms do you have?"

## 4.4 Bias Impact Analysis (Phase 1)

Preliminary prompt-based bias injections were applied to the best-performing model (Mistral-7B, 44% unbiased):

| Bias Type | Accuracy (%) | Drop from Baseline |
|---|---|---|
| None (Unbiased) | 44.0 | — |
| Recency Bias | 34.0 | −10.0 pp |
| Gender Bias | 35.0 | −9.0 pp |
| Confirmation Bias | 32.0 | −12.0 pp |

Confirmation bias caused the greatest degradation (−12 pp), as the model anchored to an initial incorrect hypothesis and selectively amplified supporting evidence while discounting contradictory symptoms.

## 4.5 Three Critical Architectural Flaws Identified

**Flaw 1: Non-Deterministic Execution:** The `while`-loop frequently failed to handle agent transitions when the LLM generated malformed stop tokens, leading to simulation crashes or silent skips. There was no formal state machine to enforce valid transitions.

**Flaw 2: No Trajectory Analysis:** Only the terminal diagnosis was evaluated. A model that guessed correctly without ordering *any* diagnostic tests received the same score as a methodical, evidence-driven clinician. The *how* of reasoning was invisible.

**Flaw 3: Lexical Leakage Vulnerability:** Due to VRAM constraints, a *homogeneous engine* was used: identical Mistral-7B weights generated both the Patient's symptoms and the Doctor's diagnosis. The Doctor could subconsciously access ground truth through the shared embedding manifold (see Chapter 1 definition).

**Why It Matters for This Thesis**

These three flaws were the *direct motivation* for building AgentClinic v2. Flaw 1 → LangGraph DCG. Flaw 2 → Metacognitive reflection node + process metrics. Flaw 3 → Heterogeneous multi-engine + NF4 quantisation.

## 4.6 Potential Viva Questions — Chapter 4

**Likely Viva Questions**

1. **Q:** Why did the generalist Mistral-7B outperform medically fine-tuned models?

   **A:** Catastrophic forgetting. BioGPT and JSL-Med were fine-tuned on static medical QA pairs, which eroded their conversational and multi-turn reasoning capabilities—skills essential for interactive simulation.

2. **Q:** Is 44% accuracy acceptable for clinical AI?

   **A:** No—it means more than half of diagnoses are wrong. However, the Phase 1 goal was *feasibility demonstration*, not deployment readiness. We showed that open-source LLMs *can* navigate multi-turn clinical conversations at all.

3. **Q:** Could the 44% be inflated by Lexical Leakage?

   **A:** Yes—this is exactly the concern. A homogeneous architecture was used due to VRAM constraints. The v2 experiment (E1) demonstrated that removing Leakage via heterogeneous engines *increased* accuracy to 45.6%, proving the scaffold matters more than the leakage.

4. **Q:** What is the difference between anchoring bias and confirmation bias?

   **A:** Anchoring creates the wrong hypothesis (over-reliance on first information). Confirmation bias *maintains* the wrong hypothesis by selectively seeking supporting evidence.

5. **Q:** Why did you test bias on only Mistral-7B?

   **A:** It was the best-performing model. Testing bias on already-failing models would not provide useful signal. The goal was to measure how bias degrades the strongest baseline.

6. **Q:** What is a while-loop orchestrator and why is it problematic?

   **A:** A simple Python loop that alternates turns between agents. It's problematic because it has no formal state management—if the LLM generates unexpected output (malformed tokens, missing stop signals), the loop has no recovery mechanism, leading to crashes or invalid states.

# 5 Chapter 5: Enhanced Methodology — AgentClinic v2

## 5.1 LangGraph Directed Cyclic Graph (DCG)

### 5.1.1 What is a State Machine?

**Academic Definition**

A **finite-state machine (FSM)** is a computational model consisting of a finite set of states, a set of transitions between states, and a set of actions. At any point, the system is in exactly one state. Transitions are triggered by inputs and governed by deterministic rules. An FSM guarantees that every possible input produces a well-defined next state—eliminating undefined behaviour.

**Simple Analogy**

A traffic light is a state machine: Green → Yellow → Red → Green. It never goes Green → Green or shows two colours at once. Every transition is predictable and deterministic. The while-loop from Phase 1 was like a traffic light with no timer—sometimes it randomly skips yellow or stays red forever.

### 5.1.2 What is LangGraph?

**Academic Definition**

**LangGraph** is a library (by LangChain) for building stateful, multi-actor LLM applications modelled as graphs. Nodes represent computation steps (agent calls), edges represent transitions, and a `TypedDict` shared state is passed between nodes. It provides built-in support for cycles, conditional routing, and state persistence.

### 5.1.3 Why a Cyclic Graph (DCG) and Not a DAG?

**Academic Definition**

A **Directed Acyclic Graph (DAG)** has no cycles—once a node is visited, it cannot be revisited. A **Directed Cyclic Graph (DCG)** permits revisitation. Clinical consultation is inherently cyclic: the doctor asks a question, receives an answer, reflects, asks another question, orders a test, reflects again, and repeats. This cycle continues for up to 10 turns.

**DCG flow:**

```
doctor_reflection -> doctor_speaker -> {patient | measurement | END}
       ^                                          |
       +--------------- Response -----------------+
```

**Why It Matters for This Thesis**

The DCG replaces the while-loop with a mathematically well-defined control structure. Every state transition is logged, every edge is deterministic, and invalid transitions are structurally impossible. This eliminates Flaw 1 (non-determinism) from Phase 1.

## 5.2 The Latent Metacognitive Reflection Node

**Academic Definition**

The `doctor_reflection` node is a hidden computation step that executes *before* every Doctor utterance. The LLM is forced to output a strictly structured JSON array of its current top-3 differential diagnoses (e.g., `["Asthma", "COPD", "Pneumonia"]`). This node operates at a low sampling temperature ($T = 0.1$) for deterministic extraction. These hypotheses are stored in the `ClinicalState` trajectory log but are **not injected** into the patient-facing dialogue history.

**Simple Analogy**

**"Think before you speak."** Imagine a doctor who writes down their top 3 guesses on a private notepad before every question they ask the patient. The patient never sees the notepad. But at the end, the examiner can review all the notepad entries to see how the doctor's thinking evolved. That's the reflection node—a private scratchpad that makes thinking visible.

**Why It Matters for This Thesis**

This is the **core innovation**. It makes the LLM's internal reasoning observable and quantifiable (enabling DS, TR, IE metrics) without contaminating the dialogue. It also acts as a Latent Chain-of-Thought scaffold—by forcing the LLM to commit to structured diagnoses first, subsequent dialogue is constrained to align with prior reasoning.

## 5.3 ClinicalState TypedDict

Every field in the shared state explained:

| Field | Purpose |
|---|---|
| `last_input_to_doctor` | The most recent message sent to the Doctor (from Patient or Measurement) |
| `last_doctor_message` | The Doctor's most recent public utterance |
| `differential_diagnoses` | Current top-3 DDx from the reflection node (JSON array) |
| `turn_count` | Current turn number (0 to max_turns) |
| `diagnosis_ready` | Boolean flag: has the Doctor declared a final diagnosis? |
| `differential_trajectory` | List of *all* DDx arrays from every reflection—the full reasoning history |
| `tests_ordered` | List of all tests requested by the Doctor (for TR computation) |
| `full_dialogue` | Complete dialogue log for post-hoc analysis |

## 5.4 Deterministic Routing via `route_doctor()`

**Academic Definition**

The `route_doctor()` function is the conditional edge function in the DCG. After `doctor_speaker` generates a response, this function inspects the output and state to determine the next node:

- If output contains `REQUEST TEST: <test_name>` → route to `measurement_node`
- If output contains `DIAGNOSIS READY: <diagnosis>` → route to `END`
- If `turn_count` ≥ `max_turns` → route to `END`
- Otherwise → route to `patient_node` (continue dialogue)

**Why It Matters for This Thesis**

This prevents hallucinated workflow violations. The LLM cannot "skip" to a diagnosis without the routing function detecting it. It cannot order a test and simultaneously talk to the patient. Every transition is validated and logged.

## 5.5 Heterogeneous Multi-Engine Architecture

**Academic Definition**

A **heterogeneous multi-engine architecture** uses **different model weights** for different agent roles. In v2: the Doctor runs on one model (e.g., Mistral-7B), while the Patient, Measurement, and Moderator run on separate, architecturally distinct model instances. This severs the shared embedding manifold, eliminating Lexical Leakage.

**VRAM budget math (NVIDIA H100 NVL, 95.8 GB):**

| Component | FP16 | NF4 + Double Quant |
|---|---|---|
| Doctor (7B) | 14 GB | ∼3.8 GB |
| Patient (7B) | 14 GB | ∼3.8 GB |
| Moderator (7B) | 14 GB | ∼3.8 GB |
| KV-Cache (3 models) | — | ∼6 GB |
| Activations + overhead | — | ∼4 GB |
| **Total** | **>42 GB + overhead** | **∼21.4 GB** |

NF4 quantisation makes the heterogeneous architecture *feasible* within a single GPU.

## 5.6 4-Bit NF4 Quantisation with Double Quantisation

**Academic Definition**

**NormalFloat4 (NF4):** Pre-trained neural network weights follow a zero-mean Gaussian distribution. NF4 creates 16 non-linearly spaced quantisation bins that are information-theoretically optimal for $\mathcal{N}(0, \sigma^2)$ data—placing more bins in the dense central region and fewer in sparse tails.

**Memory formula:**

$$\text{VRAM}_{\text{model}} \approx \text{Params} \times \left( \frac{4 \text{ bits}}{8 \text{ bits/byte}} \right) + \text{KV-Cache Overhead}$$

**Double quantisation:** Quantisation constants (one FP32 scalar per 64-weight block) are themselves quantised from FP32 to FP8, saving ≈0.37 bits per parameter.

**Simple Analogy**

**NF4:** Imagine you have 16 colour labels to describe every pixel in a photo. Uniform quantisation would spread those 16 labels evenly across all possible colours—wasting labels on rarely-used extremes (neon green, pure white). NF4 says: "Most pixels are medium-toned. Let me put 10 labels in the medium range and only 6 for extremes." It's smarter compression.

**Double quantisation:** After compressing the photo, you also compress the legend/key that explains the compression. Meta-compression.

## 5.7 Flash Attention 2

**Academic Definition**

Standard self-attention computes $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$ and materialises the full $n \times n$ attention matrix in GPU memory, requiring $O(n^2)$ memory. For $n = 4096$ tokens, this is ∼64 million entries per head per layer.

**Flash Attention 2** avoids materialising the full attention matrix by computing attention in **tiles** (blocks) that fit in GPU SRAM (fast on-chip memory). It uses online softmax computation and kernel fusion, reducing memory from $O(n^2)$ to $O(n)$ while maintaining exact mathematical equivalence.

**Simple Analogy**

Imagine multiplying two huge spreadsheets. The naive way loads both entire spreadsheets into RAM—crashing your computer. Flash Attention is like processing the multiplication one small block at a time, only loading what fits in your computer's fast cache, and streaming results out. Same answer, much less memory.

## 5.8 Cognitive Bias Injection Manifold

### 5.8.1 Recency Bias ($k = 5$ Case Memory Queue)

**Academic Definition**

The system maintains a running FIFO queue of the $k = 5$ most recently processed case summaries. These are injected into the Doctor's system prompt, mimicking a physician overly influenced by recent clinical encounters. This exploits the transformer's natural tendency to assign elevated attention weights to tokens near the current generation boundary (positional recency effect in causal attention).

### 5.8.2 Confirmation Bias (Strong Prior Hypothesis Injection)

**Academic Definition**

The Doctor is primed with a strong, **incorrect** initial hypothesis (e.g., "You are initially confident the patient has malignancy."). This warps the model's forward probability distribution toward confirmatory evidence acquisition, operationalising **premature diagnostic closure**—the model seeks evidence supporting its prior and discounts contradictions.

## 5.9 Task-Specific Dynamic LoRA Adapter Routing

This is arguably the most nuanced technical contribution of the thesis. It resolves the fundamental tension between domain specialisation and instruction alignment by decomposing the LLM's task into two independent sub-tasks, each served by a dedicated lightweight adapter.

### 5.9.1 The Problem: Why Can't We Just Fine-Tune the Whole Model?

**Academic Definition**

**Full fine-tuning** updates *all* parameters of a pre-trained model on a downstream task. For a 7B-parameter model, this involves modifying all ∼7 billion weights via gradient descent. The process requires:

- Full FP32 copies of model weights (∼28 GB), gradients (∼28 GB), and optimizer states (∼56 GB for AdamW) =∼112 GB just for a 7B model.
- Risk of **catastrophic forgetting**: the gradient updates that improve medical QA accuracy systematically overwrite the attention weight distributions that originally supported conversational coherence, multi-step logical reasoning, and instruction following.

**Simple Analogy**

Full fine-tuning is like renovating an entire house to add a new kitchen. You might get an amazing kitchen, but in the process you accidentally rip out the plumbing, rewire the electricity wrong, and break the front door. LoRA is like adding a modular kitchen island that bolts onto the existing counter—you get the new functionality without touching (or destroying) anything else in the house.

**Why It Matters for This Thesis**

In Chapter 4, we observed that medically fine-tuned models (BioGPT 35%, JSL-Med 32%) *underperformed* a generalist (Mistral-7B 44%) in interactive simulation. This is because full medical SFT destroyed their conversational abilities. LoRA avoids this by keeping the base model frozen.

### 5.9.2 What is LoRA? — Deep Dive

**Academic Definition**

**Low-Rank Adaptation (LoRA)** (Hu et al., 2022) is founded on the hypothesis that the *intrinsic dimensionality* of task-specific weight updates is much smaller than the full parameter space. In other words, fine-tuning doesn't need to modify all 7 billion parameters—a tiny, structured perturbation in the right subspace is sufficient.

Formally, for any pre-trained weight matrix $W_0 \in \mathbb{R}^{d\times k}$ (e.g., an attention projection matrix), LoRA freezes $W_0$ entirely and learns a low-rank additive update:

$$W = W_0 + \Delta W = W_0 + BA$$

where:

- $B \in \mathbb{R}^{d\times r}$ — the "up-projection" matrix (projects from rank $r$ to dimension $d$)
- $A \in \mathbb{R}^{r\times k}$ — the "down-projection" matrix (projects from dimension $k$ to rank $r$)
- $r \ll \min(d, k)$ — the **rank**, controlling adapter capacity
- $\Delta W = BA$ has rank at most $r$ — hence "low-rank adaptation"

During training, **only** $B$ and $A$ receive gradients. $W_0$ is frozen (requires no gradient, no optimizer state). This reduces trainable parameters by orders of magnitude.

**Worked Example: LoRA Parameter Count**

For a single attention projection matrix in Mistral-7B, $W \in \mathbb{R}^{4096 \times 4096}$:

**Full fine-tuning:**

$$4096 \times 4096 = 16{,}777{,}216 \text{ trainable parameters per matrix}$$

**LoRA with $r = 16$:**

$$B : 4096 \times 16 = 65{,}536 \quad + \quad A : 16 \times 4096 = 65{,}536 \quad = \quad 131{,}072 \text{ params}$$

**Compression ratio:**

$$\frac{16{,}777{,}216}{131{,}072} = 128 \times \text{ fewer parameters}$$

**Across the full model:** We apply LoRA to 4 projection matrices (Q, K, V, O) across 32 transformer layers:

$$4 \text{ matrices} \times 32 \text{ layers} \times 131{,}072 \text{ params} = 16{,}777{,}216 \text{ total LoRA params} \approx 25 \text{ MB}$$

That's 25 MB for an adapter vs 28 GB for the full model — a 1,120× reduction.

### 5.9.3 The Scaling Factor $\alpha$

**Academic Definition**

The effective LoRA update is scaled before being added to the base weights:

$$W = W_0 + \frac{\alpha}{r} \cdot BA$$

where $\alpha$ is a hyperparameter (we use $\alpha = 32$). The ratio $\frac{\alpha}{r}$ controls the **magnitude** of the adapter's influence:

- $\frac{\alpha}{r} = \frac{32}{16} = 2.0$ — the adapter's contribution is amplified by 2×
- If $\alpha = r$, the scaling is 1.0 (neutral)
- Larger $\frac{\alpha}{r}$ means the adapter has a stronger effect on the base model's behaviour

**Simple Analogy**

Think of $\alpha/r$ as the volume knob on the adapter. $r$ is the complexity of the adapter (how many musical instruments it has). $\alpha$ is how loud those instruments play. With $r = 16$ and $\alpha = 32$, the adapter plays at $2\times$ volume—loud enough to meaningfully redirect the model's behaviour, but not so loud that it drowns out the base model entirely.

### 5.9.4 Which Layers Are Modified? (Target Modules)

**Academic Definition**

In our implementation, LoRA adapters are applied to **four attention projection matrices** in every transformer layer:

| Matrix | Role |
|---|---|
| `q_proj` ($W_Q$) | Projects input into **Query** space—controls "what am I looking for?" |
| `k_proj` ($W_K$) | Projects input into **Key** space—controls "what am I offering as a match?" |
| `v_proj` ($W_V$) | Projects input into **Value** space—controls "what information do I carry?" |
| `o_proj` ($W_O$) | Projects the multi-head output back—controls "how do I combine what I found?" |

By targeting all four, the adapter can redirect *what the model attends to*, *what it matches against*, *what values it retrieves*, and *how it recombines information*—giving maximum expressiveness for minimal parameter cost.

### 5.9.5 The Two Adapters: Reasoning and Dialogue

**Academic Definition**

Two independent LoRA adapters are trained with identical hyperparameters ($r = 16$, $\alpha = 32$, target = {q, k, v, o}_proj) but on **different data distributions**:

1. **Reasoning Adapter** (`adapters/reasoning_lora`):
   - **Training data:** Clinical case presentations → structured JSON differential diagnoses
   - **Format:** Input = patient history; Output = `["Disease A", "Disease B", "Disease C"]`
   - **Purpose:** Steers the LLM toward medical diagnostic reasoning and structured JSON output
   - **Activates during:** `doctor_reflection` node (the private thinking step)
   - **Temperature:** $T = 0.1$ (deterministic, precise)
2. **Dialogue Adapter** (`adapters/dialogue_lora`):
   - **Training data:** Patient statements → empathetic, targeted doctor follow-ups
   - **Format:** Input = patient utterance; Output = empathetic 1-3 sentence response
   - **Purpose:** Preserves conversational coherence, empathy, and structured question-asking
   - **Activates during:** `doctor_speaker` node (the patient-facing dialogue)
   - **Temperature:** $T = 0.7$ (natural conversational variation)

**Simple Analogy — Extended**

Think of a doctor who puts on different "thinking hats" at different moments during a consultation:

**Hat 1 — The Medical Textbook Hat (Reasoning Adapter):**

The doctor picks up their private notepad. They put on their "textbook hat" and write:

*"Based on symptoms, my top 3 guesses are: Pneumonia, COPD, Asthma."*

The patient never sees this notepad. The hat steers the doctor toward precise, clinical thinking.

**Hat 2 — The Bedside Manner Hat (Dialogue Adapter):**

The doctor puts down the notepad and faces the patient. They switch to the "bedside manner hat" and say:

*"I understand you've been coughing for three weeks. Can you take a deep breath so I can listen to your lungs?"*

This hat steers the doctor toward warm, empathetic, conversational communication.

**Key insight:** It's the *same brain* (same frozen base model) underneath both hats. The hats are tiny (25 MB each, vs the 3.8 GB brain). Swapping hats is instant. Neither hat damages the brain—they just redirect its focus.

### 5.9.6 How Adapter Routing Works in One Turn

The following shows exactly what happens at Turn 3 of a clinical simulation:

```
Turn 3:
  Step 1: model.set_adapter("reasoning")     # Swap to reasoning hat
          doctor_reflection executes at T=0.1
          Output: ["Pneumonia", "COPD", "Asthma"]
          Stored in ClinicalState.differential_trajectory
          NOT shown to patient

  Step 2: model.set_adapter("dialogue")      # Swap to dialogue hat
          doctor_speaker executes at T=0.7
          Output: "I'd like to listen to your lungs.
                   Can you take a deep breath for me?"
          Sent to patient, stored in full_dialogue
```

```
Step 3: route_doctor() inspects output:
        No "REQUEST TEST" -> not measurement_node
        No "DIAGNOSIS READY" -> not END
        turn_count < max_turns -> route to patient_node

Step 4: Patient responds -> cycle back to Step 1
```

The adapter swap (`model.set_adapter()`) is a **pointer swap**—it changes which set of $BA$ matrices are added to the frozen $W_0$. No weights are reloaded, no VRAM is reallocated. The swap takes $< 1$ millisecond.

### 5.9.7 Training the Adapters: QLoRA Pipeline

**Academic Definition**

**QLoRA (Quantised LoRA)** trains LoRA adapters on a 4-bit NF4 quantised base model. The base model weights are frozen in NF4 (4-bit); only the adapter matrices $B$ and $A$ are trained in full BF16 (16-bit) precision. This allows fine-tuning a 7B model on as little as ∼12 GB VRAM.

**Critical design decision — avoiding data leakage:**

The adapters are trained on `agentclinic_nejm.jsonl` (New England Journal of Medicine cases), while the evaluation experiments (E1–E6) run on `agentclinic_medqa.jsonl` (MedQA scenarios). This strict train/test separation ensures the model cannot memorise evaluation answers during adapter training.

**Why It Matters for This Thesis**

If we trained on the same data we evaluate on, the 54.7% accuracy in E6 would be meaningless—the model would simply be recalling memorised answers, not demonstrating genuine reasoning improvement. By training on NEJM and evaluating on MedQA, we prove the adapters teach *generalisable skills* (how to reason, how to converse), not case-specific answers.

### 5.9.8 Why Dynamic LoRA Resolves the Trade-off

**Academic Definition**

The specialisation–alignment trade-off (Chapter 4) occurs because **monolithic weight modification** entangles domain knowledge and conversational skills in the same parameter space. Gradient updates that improve medical QA accuracy simultaneously degrade instruction-following and dialogue coherence.

Dynamic LoRA resolves this by:

1. **Decoupling:** Reasoning and dialogue are split into separate adapter subspaces that never interfere with each other.
2. **Preserving:** The frozen base model retains all its pre-trained general capabilities.
3. **Augmenting:** Each adapter adds only task-specific behaviour on top of the preserved base.

**Result:** E6 (54.7%) surpasses both E2 (50.2%, domain knowledge only) *and* E3 (53.5%, instruction alignment only), achieving the best of both worlds.

## 5.10 Theoretical Underpinnings

### 5.10.1 Decoupling Reasoning from Generation (Latent CoT)

**Academic Definition**

Standard autoregressive generation: $P(w_t \mid w_1, \ldots, w_{t-1})$. When forced to simultaneously generate dialogue *and* reason medically, self-attention heads must split capacity between syntactic fluency and semantic medical deduction. The reflection node addresses this by injecting high-information medical tokens (structured JSON DDx) into the context window *before* dialogue generation. The `doctor_speaker`'s attention heads then attend heavily to this JSON, constraining dialogue to align with prior reasoning.

### 5.10.2 Embedding Manifold Overlap and Lexical Leakage

When $\theta_{\text{Doctor}} = \theta_{\text{Patient}}$, both agents share $f_\theta$. Patient symptom embeddings and Doctor disease embeddings occupy the same manifold $\Rightarrow$ zero-shot knowledge transfer through geometric proximity. The heterogeneous architecture severs this: $\theta_{\text{Doctor}} \neq \theta_{\text{Patient}}$ forces genuine semantic deduction.

### 5.10.3 Information-Theoretic Basis of NF4

Pre-trained weights $\sim \mathcal{N}(0, \sigma^2)$. Uniform INT4 wastes bits on low-density tails. NF4 bins are placed at quantiles of the normal CDF, maximising mutual information $I(W; \hat{W})$ between the original weights $W$ and their quantised representations $\hat{W}$.

## 5.11 Potential Viva Questions — Chapter 5

**Likely Viva Questions**

1. **Q:** Why a cyclic graph and not a DAG?

   **A:** Clinical consultation is inherently iterative—the doctor asks, receives, reflects, asks again. A DAG cannot revisit nodes, so it cannot model this cycle.

2. **Q:** What happens in the reflection node?

   **A:** The LLM outputs a JSON array of top-3 differential diagnoses at $T = 0.1$. This is stored in the trajectory log but never shown to the patient.

3. **Q:** Explain LoRA in mathematical terms.

   **A:** LoRA decomposes the weight update $\Delta W$ into $BA$ where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$ with $r = 16 \ll 4096$. This reduces trainable parameters by 128× per matrix. The frozen base model $W_0$ is never modified, so pre-trained capabilities are preserved.

4. **Q:** What is $\alpha$ in LoRA and what does it control?

   **A:** $\alpha$ is a scaling factor. The effective update is $\frac{\alpha}{r} \cdot BA$. With $\alpha = 32$ and $r = 16$, the scaling is 2×. This controls how strongly the adapter modifies the base model's behaviour—like a volume knob.

5. **Q:** Why do you target Q, K, V, and O projections?

   **A:** These four matrices control the entire attention mechanism: what to look for (Q), what to match (K), what to extract (V), and how to combine (O). Targeting all four gives maximum adapter expressiveness.

6. **Q:** How does LoRA avoid catastrophic forgetting?

   **A:** By freezing all 7 billion base parameters. Only the adapter matrices (∼25 MB) receive gradients. The base model's conversational, reasoning, and instruction-following abilities are completely preserved.

7. **Q:** Why two separate adapters instead of one?

   **A:** Because reasoning (structured medical JSON) and dialogue (empathetic conversation) are fundamentally different tasks. A single adapter would face the same entanglement problem as full fine-tuning. Separate adapters let each task optimise independently.

8. **Q:** Why did you train on NEJM data instead of MedQA data?

   **A:** To avoid data leakage. If the adapter were trained on the same MedQA scenarios used for evaluation, it would memorise answers rather than learning generalisable reasoning skills. Training on NEJM and evaluating on MedQA proves the adapter teaches transferable clinical competence.

9. **Q:** What is the VRAM overhead of the dual LoRA adapters?

   **A:** Approximately 50 MB combined (25 MB each)—negligible compared to

# 6 Chapter 6: Experiments and Results

## 6.1 Experimental Design

Six experiments were conducted over **107 AgentClinic-MedQA scenarios**, capped at **10 multi-turn inference steps** per scenario, on a single NVIDIA H100 NVL GPU (95.8 GB VRAM).

| Exp. | Doctor Engine | Bias | NF4 |
|---|---|---|---|
| E1 | Mistral-7B-Instruct-v0.2 | None | Off |
| E2 | JSL-MedMistral-7B-v2.2 | None | Yes |
| E3 | Llama-3.1-8B-Instruct | None | Yes |
| E4 | JSL-MedMistral-7B-v2.2 | Recency | Yes |
| E5 | JSL-MedMistral-7B-v2.2 | Confirmation | Yes |
| E6 | JSL-MedMistral-7B + LoRA | None | Yes |

## 6.2 Full Results Table

| Exp | Doctor | Bias | Acc% | DS | TR | IE |
|---|---|---|---|---|---|---|
| E1 | Mistral-7B | None | 45.6 | 0.56 | 0.63 | 1.13 |
| E2 | JSL-MedMistral | None | 50.2 | 0.68 | 0.71 | 0.98 |
| E3 | Llama-3.1-8B | None | 53.5 | 0.65 | 0.79 | 1.05 |
| E4 | JSL-MedMistral | Recency | 38.1 | 0.45 | 0.58 | 1.34 |
| E5 | JSL-MedMistral | Confirmation | 31.4 | 0.88 | 0.41 | 0.61 |
| E6 | JSL-Med + LoRA | None | **54.7** | 0.72 | 0.76 | 1.02 |

## 6.3 Metric Definitions with Worked Examples

### 6.3.1 Diagnostic Stability (DS)

**Academic Definition**

DS measures the average Jaccard similarity between consecutive differential diagnoses over $T$ turns. It captures how consistently a model refines its hypotheses (vs. erratic switching).

$$DS = 1 - \frac{1}{T-1}\sum_{t=1}^{T-1}\left(1 - \frac{|DDx_t \cap DDx_{t+1}|}{|DDx_t \cup DDx_{t+1}|}\right) = \frac{1}{T-1}\sum_{t=1}^{T-1} J(DDx_t, DDx_{t+1})$$

where $J(A,B) = \frac{|A \cap B|}{|A \cup B|}$ is the Jaccard similarity coefficient.

**Worked Example: Jaccard Similarity for DS**

Suppose a 4-turn scenario produces these differentials:

- Turn 1: {Asthma, COPD, Pneumonia}
- Turn 2: {Asthma, COPD, Bronchitis}
- Turn 3: {COPD, Bronchitis, Lung Cancer}
- Turn 4: {COPD, Bronchitis, Lung Cancer}

**Step 1:** $J(T1, T2) = \frac{|\{\text{Asthma, COPD}\}|}{|\{\text{Asthma, COPD, Pneumonia, Bronchitis}\}|} = \frac{2}{4} = 0.50$

**Step 2:** $J(T2, T3) = \frac{|\{\text{COPD, Bronchitis}\}|}{|\{\text{Asthma, COPD, Bronchitis, Lung Cancer}\}|} = \frac{2}{4} = 0.50$

**Step 3:** $J(T3, T4) = \frac{|\{\text{COPD, Bronchitis, Lung Cancer}\}|}{|\{\text{COPD, Bronchitis, Lung Cancer}\}|} = \frac{3}{3} = 1.00$

**DS** $= \frac{0.50+0.50+1.00}{3} = 0.667$

**Interpretation:** Moderate stability. The model refined hypotheses gradually (good), with the final two turns showing complete convergence.

**What does DS = 0.56 vs. DS = 0.88 mean?**

- $DS = 0.56$ (E1): Frequent hypothesis revisions. The model changes its differential frequently—potentially reactive and exploratory, but also possibly noisy.
- $DS = 0.88$ (E5): Almost no changes across turns. The model stubbornly maintains its initial hypothesis—which sounds good, but is **dangerous when combined with low accuracy (31.4%)**. This is premature diagnostic closure.

### 6.3.2 Test Rationality (TR)

**Academic Definition**

TR is a binary compliance indicator: did the model order at least one basic exploratory test (CBC, vitals, basic metabolic panel) *before* requesting expensive or invasive procedures (MRI, biopsy, CT angiography)?

$$TR = \begin{cases} 0 & \text{if } \exists\, t_{\text{adv}} \text{ ordered before any } t_{\text{basic}} \\ 1 & \text{otherwise} \end{cases}$$

**Worked Example: Test Ordering for TR**

**Rational ordering (TR = 1):**

1. Vital signs check (*basic*)
2. Complete Blood Count (*basic*)
3. Chest X-ray (*intermediate*)
4. CT scan (*advanced*)

**Irrational ordering (TR = 0):**

1. MRI Brain (*advanced*) — ordered before *any* basic test!
2. Vital signs check (*basic*)

The second sequence fails TR because the model jumped to an expensive, invasive test without first gathering basic information.

### 6.3.3 Information Efficiency (IE)

**Academic Definition**

IE measures the ratio of unique differential hypotheses generated ($N_{\text{unique}}$) to the total number of dialogue turns ($T$):

$$IE = \frac{N_{\text{unique}}}{T}$$

$IE \approx 1.0$: Each turn produces a new hypothesis (efficient exploration).

$IE \gg 1.0$: Over-testing—gathering more data points than hypotheses warrant.

$IE \ll 1.0$: Under-exploration—failing to generate new hypotheses from available evidence.

**Why is E4's IE = 1.34 paradoxically high?**

Under recency bias, the model panics and orders many tests per turn, gathering excess data without meaningfully refining its differential. It over-tests due to "bias panic"—each injected case memory triggers a new round of defensive testing.

## 6.4 Five Key Findings Explained

### 6.4.1 Finding 1: E1 Accuracy (45.6%) Increased from Phase 1 (44%)

**Academic Definition**

Despite *removing* Lexical Leakage by switching to heterogeneous engines, accuracy *increased* by 1.6 percentage points. This proves that the LangGraph DCG + metacognitive reflection node provides a reasoning scaffold **more powerful** than the implicit performance boost from shared embedding manifolds. Structured metacognition eclipses Lexical Leakage.

### 6.4.2 Finding 2: Why E3 > E2 (Instruction Alignment vs. Domain Fine-Tuning)

Llama-3.1-8B (E3: 53.5%) outperformed JSL-MedMistral (E2: 50.2%) despite having *less* medical domain knowledge. Llama-3.1's superior **instruction alignment** gave it better command following ($TR = 0.79$ vs. 0.71), better multi-turn coherence, and better conditional routing compliance. In interactive clinical simulation, *how well you follow instructions* matters as much as *what you know about medicine*.

### 6.4.3 Finding 3: E4 Recency Bias — "Hypothesis Thrashing"

The $k = 5$ injected case memories destabilise the Doctor's reasoning. $DS = 0.45$ (lowest stability) means the model constantly rewrites its differential to match recent cases rather than the current patient's evidence. Accuracy drops to 38.1%—the model conflates independent patients.

### 6.4.4 Finding 4: E5 Confirmation Bias — "Premature Diagnostic Closure"

The strong prior hypothesis makes the model appear *very stable* ($DS = 0.88$, highest in the study) while being *catastrophically wrong* (31.4%, lowest accuracy). The model stubbornly refuses to update its differential, ignoring contradictory evidence from the Measurement Agent. This is the **most dangerous failure mode**: confident consistency with systematic incorrectness.

### 6.4.5 Finding 5: E6 Dynamic LoRA — Highest Accuracy

The dual-adapter approach resolved the specialisation–alignment trade-off:

- Accuracy: 50.2% → 54.7% (+4.5 pp over E2 base)
- DS: 0.68 → 0.72 (more stable reasoning)
- TR: 0.71 → 0.76 (better test ordering)
- IE: 0.98 → 1.02 (maintained efficiency)
- VRAM overhead: only ∼50 MB

## 6.5 Potential Viva Questions — Chapter 6

**Likely Viva Questions**

1. **Q:** How is Diagnostic Stability computed? Walk me through an example.

   **A:** DS is the average Jaccard similarity between consecutive DDx sets. [Provide worked example above.]

2. **Q:** Why is high DS with low accuracy dangerous?

   **A:** It means the model is *confidently wrong*—it never changes its mind despite contradictory evidence. This is premature diagnostic closure, the most dangerous clinical failure mode.

3. **Q:** Why did removing Lexical Leakage *increase* accuracy?

   **A:** The structural scaffold (DCG + reflection node) was more valuable than the Leakage boost. Forcing "think before you speak" improved reasoning quality beyond what shared embeddings provided.

4. **Q:** What does IE = 1.34 mean in E4?

   **A:** The model generated more unique data points per turn than hypotheses warranted—over-testing due to recency bias "panic." The injected case memories made the model defensively order extra tests.

5. **Q:** How did LoRA improve accuracy by 4.5 points?

   **A:** The reasoning adapter anchored the reflection node's DDx to medical evidence; the dialogue adapter preserved conversational coherence. Decoupling these tasks at the adapter level prevented the attention head "tug-of-war" that degrades both reasoning and dialogue quality.

6. **Q:** Why does E6 surpass even E3 (Llama-3.1)?

   **A:** E3 has better instruction alignment but less medical knowledge. E6 combines JSL-MedMistral's domain expertise with LoRA-separated reasoning and dialogue, achieving the best of both worlds.

# 7 Chapter 7: Discussion

## 7.1 Structured Metacognition Eclipses Lexical Leakage

**Academic Definition**

**This is the most important finding of the thesis.** The Phase 1 homogeneous system achieved 44.0% accuracy with Lexical Leakage potentially inflating results. The Phase 2 heterogeneous system *removed* Leakage entirely, yet accuracy **rose to 45.6%**. The architectural scaffold—LangGraph DCG + latent reflection node—provided a reasoning advantage that more than compensated for the loss of any implicit embedding-manifold hints.

**Mechanistic explanation:** By forcing the LLM to populate a structured JSON differential *before* emitting dialogue, the framework:

1. Stabilises the clinical trajectory (prevents iterative hallucinations)
2. Channels attention capacity toward evidence-driven deduction
3. Creates a Latent Chain-of-Thought scaffold that constrains subsequent dialogue

**Simple Analogy**

Imagine two students taking an exam. Student A shares a brain with the answer key (Lexical Leakage) but writes answers stream-of-consciousness. Student B has no access to the answer key but is forced to write an outline of their reasoning before each answer. Student B scores *higher* because the discipline of outlining improves their thinking more than the leaked hints helped Student A. Structured thinking beats cheating.

**Why It Matters for This Thesis**

This finding validates the core thesis hypothesis: that process-level architectural interventions matter more than raw model capability or information leakage. It justifies the entire v2 redesign.

## 7.2 Domain Specialisation vs. Instruction Alignment

**Academic Definition**

A fundamental tension exists in medical AI:

- **Domain specialisation** (via medical SFT) improves terminology recall, pathological anchoring, and clinical vocabulary but risks catastrophic forgetting of general reasoning.
- **Instruction alignment** (via RLHF/DPO on general instruction data) preserves multi-step logical reasoning, command following, and conversational coherence.

Evidence: JSL-MedMistral (E2: 50.2%) has superior medical knowledge but lower TR (0.71). Llama-3.1 (E3: 53.5%) has superior instruction following and higher TR (0.79). E3 outperforms E2 by 3.3 percentage points.

**Implication:** For interactive clinical AI, *instruction alignment and logical deduction* are as critical as—or more critical than—specialty medical knowledge retrieval. The optimal clinical agent requires both.

**Simple Analogy**

A medical specialist who knows every disease but can't have a coherent conversation is less useful than a well-trained general practitioner who asks the right questions in the right order. Knowledge without process is like a library without a librarian.

## 7.3 Dynamic LoRA Resolves the Trade-off

**Academic Definition**

The E6 experiment proves that the specialisation–alignment trade-off is **not inherent to LLM architecture** but is a consequence of **monolithic weight modification**. By deploying task-specific LoRA adapters that activate conditionally:

- The **reasoning adapter** steers the model toward medical differential enumeration during reflection (domain specialisation without forgetting)
- The **dialogue adapter** maintains conversational coherence during patient interaction (instruction alignment preserved)
- Neither adapter catastrophically modifies the base model's pre-trained weights

Result: E6 (54.7%) surpasses both E2 (50.2%) and E3 (53.5%), demonstrating that you can have both domain expertise *and* instruction alignment simultaneously through adapter decomposition.

**Why It Matters for This Thesis**

This resolves the central question from Chapter 4: "Why did medical fine-tuning hurt performance?" The answer is not that medical knowledge is harmful, but that *monolithic* fine-tuning entangles reasoning and dialogue capabilities. LoRA disentangles them.

## 7.4 Safety Threat of Cognitive Biases

**Academic Definition**

E5 (confirmation bias) represents the **most dangerous failure mode identified**:

- $DS = 0.88$ (appears very stable and consistent)
- Accuracy = 31.4% (systematically wrong)

This means the model **appears confidently consistent while being catastrophically incorrect**. In a clinical deployment, a system that rarely changes its mind would be perceived as "reliable" by healthcare workers—but would be systematically misdiagnosing patients. This is a direct patient safety threat.

**Recency bias (E4)** is less dangerous in one sense ($DS = 0.45$ makes erratic behaviour visible) but still harmful: the model conflates independent patient presentations with recently seen cases.

**Simple Analogy**

**Confirmation bias (E5):** A broken compass that always points north. It seems very consistent and reliable—until you realise you're lost. High stability + low accuracy = confident wrongness.

**Recency bias (E4):** A GPS that keeps trying to reroute you to wherever you went yesterday. It's clearly struggling (low stability), but at least you can tell something is wrong.

**Why It Matters for This Thesis**

Binary accuracy alone would flag E5 as "poor" (31.4%). But it would *not* reveal that the model is dangerously confident in its wrong answers. Only the DS metric exposes this. This validates the thesis argument that process metrics are essential for clinical AI safety.

## 7.5 Implications for Clinical AI Evaluation Standards

**Academic Definition**

Binary diagnostic accuracy is an **insufficient proxy** for clinical AI safety. This thesis demonstrates that:

- A model with 45.6% accuracy and $TR < 0.5$ (ordering MRIs before vital signs) is clinically unacceptable regardless of its accuracy.
- A model with $DS < 0.4$ (erratically switching between unrelated diagnoses) is unreliable regardless of final outcome.
- A model with $DS = 0.88$ and accuracy = 31.4% is **actively dangerous**—it looks reliable but is systematically wrong.

**Recommendation:** Future clinical AI evaluation standards should mandate process-level reporting (DS, TR, IE) alongside outcome accuracy.

## 7.6 Limitations

1. **Text-only measurement agent:** Real diagnostics rely on visual data (ECG waveforms, X-rays, histology). The current system operates entirely in text.

2. **Single dataset:** All experiments use AgentClinic-MedQA (107 scenarios). Generalisation to MIMIC-IV (real EHR data) or NEJM case challenges is untested.

3. **Moderator accuracy:** The LLM-mediated semantic comparison may itself introduce evaluation noise.
4. **LoRA adapters are thesis-specific:** The reasoning and dialogue adapters require task-specific training data and may not generalise to other clinical simulation formats.

## 7.7 Potential Viva Questions — Chapter 7

**Likely Viva Questions**

1. **Q:** What is your single most important finding?

   **A:** That structured metacognition (the reflection node) eclipses Lexical Leakage. Removing the shared embedding advantage while adding forced reasoning *increased* accuracy, proving architecture matters more than model-level information flow.

2. **Q:** Why is E5 more dangerous than E4?

   **A:** E5 (confirmation bias) appears stable ($DS = 0.88$) while being wrong (31.4%). E4 (recency bias) is visibly erratic ($DS = 0.45$). Visible failure is less dangerous than invisible failure because visible failure can be detected and corrected.

3. **Q:** How does LoRA resolve the specialisation–alignment trade-off?

   **A:** By decoupling reasoning and dialogue into separate adapters. The base model is never monolithically modified; task-specific behaviours are added through lightweight, independently trained low-rank updates.

4. **Q:** Is 54.7% accuracy clinically acceptable?

   **A:** No. It means nearly half of diagnoses are wrong. However, this thesis is about *evaluation methodology*, not deployment. We demonstrate that process metrics reveal critical safety information that accuracy alone misses.

5. **Q:** What is the main limitation of your work?

   **A:** The text-only measurement agent. Real clinical diagnosis depends heavily on visual data (X-rays, ECGs, pathology). Integrating Vision-Language Models is the most important future direction.

6. **Q:** Why should binary accuracy not be the primary metric?

   **A:** Because it masks *how* the model reasons. A model that guesses correctly by luck gets the same score as one that follows proper clinical protocol. Our process metrics (DS, TR, IE) reveal whether the model reasons safely.

7. **Q:** Could your findings generalise beyond AgentClinic-MedQA?

   **A:** The architectural findings (DCG, reflection node, LoRA routing) are framework-agnostic. The specific accuracy numbers may vary on other datasets, but the principle that structured metacognition improves LLM clinical reasoning should generalise.

# 8 Chapter 8: Conclusion and Future Work

## 8.1 Summary of Findings (5 Bullet Points)

1. **Structured metacognition eclipses Lexical Leakage:** The LangGraph DCG + reflection node improved baseline accuracy from 44.0% to 45.6% *despite* removing the shared embedding advantage, proving that architectural discipline is more valuable than implicit information flow.

2. **Domain fine-tuning causes catastrophic forgetting in interactive settings:** BioGPT (35%) and JSL-Med (32%) underperformed generalist Mistral-7B (44%) in Phase 1 due to SFT destroying multi-turn conversational capabilities.

3. **Instruction alignment outperforms domain specialisation for interactive tasks:** Llama-3.1-8B (E3: 53.5%) surpassed JSL-MedMistral (E2: 50.2%) due to superior instruction following and multi-step logical reasoning.

4. **Dynamic LoRA resolves the specialisation–alignment trade-off:** Dual adapters for reasoning ($T_{\text{reflect}}$) and dialogue ($T_{\text{speak}}$) achieved the highest accuracy (E6: 54.7%) with negligible VRAM overhead (∼50 MB).

5. **Cognitive biases are a critical safety threat:** Confirmation bias induces premature diagnostic closure ($DS = 0.88$, accuracy = 31.4%)—the model appears reliable while being systematically wrong. Process metrics (DS, TR, IE) are essential to expose this.

## 8.2 Future Work

### 8.2.1 Process Reward Modelling via Direct Preference Optimisation (DPO)

**Academic Definition**

**Direct Preference Optimisation (DPO)** is a post-training alignment method that replaces the reward model + PPO pipeline of traditional RLHF with a single classification-style loss function. Given pairs of preferred ($y_w$) and dispreferred ($y_l$) outputs for the same prompt $x$, DPO directly optimises the policy $\pi_\theta$:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)} \left[ \log \sigma \left( \beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

where $\sigma$ is the sigmoid function, $\pi_{\text{ref}}$ is the frozen reference policy, and $\beta$ controls divergence from the reference.

**How it differs from RLHF:**

- RLHF: Train reward model → run PPO to maximise reward → complex, unstable, expensive.
- DPO: Skip the reward model entirely. Directly use preference pairs to shift the policy. Simpler, more stable, single-stage.

**Simple Analogy**

**RLHF:** You hire a food critic (reward model) to taste dishes and give scores. Then you train the chef (LLM) to maximise the critic's scores via trial-and-error (PPO). Expensive: you need the critic available at all times.

**DPO:** Instead of hiring a permanent critic, you just give the chef *pairs*: "This dish is better than that dish." The chef directly learns from the comparisons. No need for a separate critic.

**Application to this thesis:**

Generate trajectory pairs from AgentClinic v2:

- **Path A (rejected):** Premature hypothesis commitment, low TR, erratic DS — the model guesses early and skips tests.
- **Path B (chosen):** Systematic basic-before-advanced investigation, high DS, evidence-driven diagnosis.

The `ClinicalTrajectoryEvaluator` would score both paths, and DPO would train the

model to *prefer* Path B trajectories.

### 8.2.2 RL from AI Feedback via Self-Play

**Academic Definition**

Use AgentClinic v2 as a **Reinforcement Learning environment**. Heterogeneous agents simulate thousands of clinical cases via self-play. The `ClinicalTrajectoryEvaluator` serves as the programmatic reward function:

$$R = w_1 \cdot \text{Accuracy} + w_2 \cdot DS + w_3 \cdot TR + w_4 \cdot IE$$

Trajectories scoring highly on the composite reward are archived, and the Doctor model is iteratively fine-tuned on its own successful reasoning paths using PPO or DPO—creating a **self-improving clinical agent** without human annotation overhead.

**Simple Analogy**

Like a chess AI (AlphaZero) that plays against itself millions of times, learning from its wins and losses. The AgentClinic Doctor plays against the Patient and Measurement agents, learns which reasoning strategies lead to correct diagnoses, and gradually improves—no human expert needed.

### 8.2.3 Extending LoRA to Bias Resilience

**Academic Definition**

Train **debiasing adapters** that specifically counteract cognitive bias failure modes:

- **Recency-debiasing adapter:** Trained on trajectory pairs where the model successfully resisted anchoring to injected $k = 5$ case memories.
- **Confirmation-debiasing adapter:** Trained to maintain high TR even under strong prior hypotheses.

Combined with the existing reasoning and dialogue adapters, this creates a **four-adapter routing manifold**:

| Active Node / Condition | Adapter |
|---|---|
| `doctor_reflection` | Reasoning adapter |
| `doctor_speaker` | Dialogue adapter |
| Recency bias detected | Recency-debiasing adapter |
| Confirmation bias detected | Confirmation-debiasing adapter |

The LangGraph orchestrator dynamically selects adapters based on both the active node *and* detected bias risk.

### 8.2.4 Multimodal Integration

**Academic Definition**

The current `measurement_node` returns text-only results. Real clinical diagnosis relies heavily on visual pattern recognition:

- **ECG waveforms** (ST-elevation detection)
- **Chest X-rays** (pneumonia infiltrates, cardiomegaly)
- **Histopathology slides** (malignancy grading)
- **Dermatology images** (lesion classification)

Future work will integrate Vision-Language Models (VLMs):

- **LLaVA-Med:** Open-source biomedical VLM fine-tuned on medical image–text pairs.
- **Med-Gemini:** Google's multimodal medical AI capable of processing images, text, and genomic data.

The Measurement Agent would return raw images instead of text descriptions, and the Doctor agent (running a VLM) would interpret them directly—bridging the final gap toward complete clinical simulation fidelity.

**Simple Analogy**

Currently, the system is like a phone consultation where the doctor can only *hear* about test results ("Your X-ray shows a shadow in the left lung"). Future versions would be like a video consultation where the doctor can actually *see* the X-ray and interpret it themselves.

## 8.3 Potential Viva Questions — Chapter 8

**Likely Viva Questions**

1. **Q:** What is DPO and how does it differ from RLHF?

   **A:** DPO skips the reward model and PPO. It directly optimises the policy using preference pairs $(y_w, y_l)$ with a classification-style loss. It's simpler, more stable, and single-stage.

2. **Q:** How would you generate trajectory pairs for DPO?

   **A:** Run AgentClinic v2 on many cases. Use ClinicalTrajectoryEvaluator to score trajectories on DS, TR, IE. High-scoring trajectories become $y_w$ (chosen); low-scoring become $y_l$ (rejected).

3. **Q:** What is self-play in the context of your framework?

   **A:** The Doctor, Patient, and Measurement agents simulate thousands of cases autonomously. The reward function uses our process metrics. The Doctor is iteratively fine-tuned on successful trajectories.

4. **Q:** How would a four-adapter routing manifold work?

   **A:** The LangGraph orchestrator selects adapters based on both the active node (reflection vs. dialogue) and detected bias conditions. For example, during reflection under detected recency bias risk, both the reasoning adapter and recency-debiasing adapter are active.

5. **Q:** Why is multimodal integration the most important future direction?

   **A:** Because real diagnosis is inherently multimodal. A system that can reason about chest X-rays, ECGs, and lab values simultaneously is far closer to genuine clinical competence than a text-only system.

6. **Q:** What would be needed to make AgentClinic clinically deployable?

   **A:** (1) Multimodal measurement integration, (2) accuracy >80% on diverse datasets, (3) process metrics consistently high ($DS > 0.7$, $TR > 0.9$), (4) bias resilience via debiasing adapters, (5) clinical validation with real physicians.

7. **Q:** Summarise your thesis in three sentences.

   **A:** We built AgentClinic v2, a deterministic multi-agent framework that evaluates LLMs through interactive clinical simulations. We proved that structured metacognition (forced reflection) eclipses Lexical Leakage and that domain expertise and instruction alignment can coexist through dynamic LoRA adapter routing. We established that process-level metrics are essential for clinical AI safety, as binary accuracy alone masks dangerous failure modes like premature diagnostic closure.

# 9 Appendix A: Complete Code Workflow

This appendix traces the **entire execution flow** of `agentic_clinic_future_scope.py` from command-line invocation to final JSONL output. Understanding this flow is critical for viva preparation, as examiners often ask "walk me through what happens when you run an experiment."

## 9.1 High-Level Architecture Overview

The codebase is organised into 9 sections:

| Sec. | Name | Purpose |
|---|---|---|
| 1 | Utility Functions | String cleaning, JSON extraction, test name parsing |
| 2 | Model Inference Layer | Unified dispatcher for OpenAI / Claude / Replicate / HuggingFace |
| 3 | LocalServerLLMWrapper | NF4 quantisation, Flash Attention 2, Dynamic LoRA loading |
| 4 | ClinicalTrajectoryEvaluator | DS, TR, IE metric computation |
| 5 | Dataset Classes | `ScenarioMedQA` + `ScenarioLoaderMedQA` |
| 6 | Agent Classes | `PatientAgent`, `DoctorAgent`, `MeasurementAgent` |
| 7 | LangGraph State Machine | `ClinicalState` TypedDict definition |
| 8 | Main Execution Pipeline | Model loading, graph building, scenario loop |
| 9 | CLI Entry Point | `argparse` configuration |

## 9.2 Step-by-Step Execution Flow

### 9.2.1 Step 1: CLI Invocation and Argument Parsing (Section 9)

```
python agentic_clinic_future_scope.py \
  --doctor_llm HF_mistralai/Mistral-7B-Instruct-v0.2 \
  --patient_llm HF_mistralai/Mistral-7B-Instruct-v0.2 \
  --doctor_model_id mistralai/Mistral-7B-Instruct-v0.2 \
  --patient_model_id mistralai/Mistral-7B-Instruct-v0.2 \
  --num_scenarios 107 \
  --doctor_bias None
```

**What Happens**

`argparse` parses command-line flags:

- `-doctor_llm`: Backend identifier (prefix `HF_` → local HuggingFace model)
- `-doctor_model_id`: Actual HuggingFace model ID for weight loading
- `-patient_model_id`: Separate model for heterogeneous architecture
- `-doctor_bias`: `None` / `recency` / `confirmation`
- `-num_scenarios`: How many of the 107 cases to run

These arguments flow into the `main()` function.

### 9.2.2 Step 2: Heterogeneous Engine Loading (Section 3 via Section 8)

**What Happens**

`main()` instantiates `LocalServerLLMWrapper` for each agent role:

**2a. Doctor Engine:**

1. Load tokenizer from HuggingFace: `AutoTokenizer.from_pretrained(model_id)`
2. Configure NF4 quantisation: `BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4", bnb_4bit_use_double_quant=True)`
3. Load model: `AutoModelForCausalLM.from_pretrained(model_id, quantization_config=bnb_config)`
4. Attempt Flash Attention 2: `attn_implementation="flash_attention_2"` (silent fallback if unavailable)
5. Check for LoRA adapters at `adapters/reasoning_lora` and `adapters/dialogue_lora`
6. If found: wrap model with `PeftModel.from_pretrained()` and load both adapters
7. Create text-generation pipeline: `pipeline("text-generation", model=model, tokenizer=tokenizer)`

**2b. Patient Engine:**

- If `patient_model_id == doctor_model_id`: share the Doctor engine (homogeneous mode — Lexical Leakage risk!)
- If different: load a separate `LocalServerLLMWrapper` instance (heterogeneous mode — Leakage eliminated)

**2c. Moderator Engine:**

- Shares the Patient engine by default, or loads a separate one if specified

**Why It Matters**

This is where the heterogeneous multi-engine architecture is enforced. If doctor and patient share the same engine, $\theta_{\text{Doctor}} = \theta_{\text{Patient}}$ and Lexical Leakage is possible. If they are different engines, the embedding manifolds are severed.

### 9.2.3 Step 3: Dataset Loading (Section 5)

**What Happens**

`ScenarioLoaderMedQA` reads `agentclinic_medqa.jsonl` line-by-line:

1. Each line is parsed as a JSON object
2. Wrapped in a `ScenarioMedQA` object that partitions data:
   - `.examiner_information()` → Doctor sees the objective
   - `.patient_information()` → Patient holds the symptoms
   - `.exam_information()` → Measurement holds test results
   - `.diagnosis_information()` → Moderator holds ground truth

This is the **knowledge partitioning** that makes the simulation realistic.

### 9.2.4 Step 4: Scenario Loop (Section 8, repeated 107 times)

For each scenario, the following happens:

**4a. Agent Instantiation (Section 6):**

- `DoctorAgent(scenario, bias=doctor_bias, recent_cases=[...])` — receives only the `Objective_for_Doctor`
- `PatientAgent(scenario, bias=patient_bias)` — receives only the `Patient_Actor` symptoms
- `MeasurementAgent(scenario)` — receives only the `Test_Results` and `Physical_Examination_F`

**4b. Bias Injection:**

- If `doctor_bias == "recency"`: the last $k = 5$ case summaries are injected into the Doctor's system prompt
- If `doctor_bias == "confirmation"`: a strong prior hypothesis ("You are initially confident the patient has cancer") is injected

### 9.2.5 Step 5: LangGraph Construction (Section 7 + Section 8)

**What Happens**

The directed cyclic graph is built:

1. Create `StateGraph(ClinicalState)` — defines the shared state schema
2. Add 4 nodes:
   - `doctor_reflection` — hidden metacognitive reasoning
   - `doctor_speaker` — patient-facing dialogue
   - `patient` — patient response
   - `measurement` — test result delivery
3. Set entry point: `doctor_reflection`
4. Add deterministic edges:
   - `doctor_reflection` → `doctor_speaker` (always)
   - `patient` → `doctor_reflection` (always)
   - `measurement` → `doctor_reflection` (always)
5. Add conditional edge from `doctor_speaker` via `route_doctor()`:
   - "DIAGNOSIS READY" in output OR turn $\geq$ max → `END`
   - "REQUEST TEST" in output → `measurement`
   - Otherwise → `patient`
6. Compile: `app = workflow.compile()`

### 9.2.6 Step 6: Graph Execution (One Full Consultation)

**What Happens**

`app.invoke(initial_state)` starts the state machine. Here is one complete cycle:

**Initial State:**

```
{
  "last_input_to_doctor": "Please begin the examination.",
  "turn_count": 0,
  "diagnosis_ready": False,
  "differential_trajectory": [],
  "tests_ordered": [],
  "full_dialogue": []
}
```

**Cycle: Turn 1**

1. **doctor_reflection node:**
   - If LoRA available: `model.set_adapter("reasoning")`
   - System prompt: "List top 3 differential diagnoses as JSON array"
   - Temperature: $T = 0.1$
   - Output: `["Pneumonia", "Asthma", "Bronchitis"]`
   - State update: `differential_diagnoses` and `differential_trajectory` updated
   - This output is **NOT** shown to the patient
2. **doctor_speaker node:**
   - If LoRA available: `model.set_adapter("dialogue")`
   - System prompt includes: turn count, bias injection (if any), presentation info
   - Temperature: $T = 0.7$
   - Output: "Good morning. I'd like to ask about your cough. When did it start?"
   - State update: `last_doctor_message`, `turn_count` incremented, `full_dialogue` appended
3. **route_doctor() evaluates:**
   - No "DIAGNOSIS READY" → not END
   - No "REQUEST TEST" → not measurement
   - → route to `patient`
4. **patient node:**
   - Receives doctor's message, generates response using Patient engine
   - Output: "It started about three weeks ago. I've been coughing up yellowish phlegm."

### 9.2.7 Step 7: Evaluation (Moderator + Metrics)

**What Happens**

**7a. Semantic Diagnosis Comparison:**

- Extract predicted diagnosis from `final_state["last_doctor_message"]`
- Call `compare_results(predicted, ground_truth, moderator_engine)`
- Moderator LLM compares semantically: "Community-Acquired Pneumonia" vs "Pneumonia" → "Yes" (correct)

**7b. Process Metrics via ClinicalTrajectoryEvaluator (Section 4):**

- **Diagnostic Stability:** Average Jaccard similarity across `differential_trajectory`
- **Test Rationality:** Check if any advanced test was ordered before a basic test in `tests_ordered`
- **Information Efficiency:** $\frac{\text{tests_ordered count}+\text{turn_count}}{\text{turn_count}}$

### 9.2.8 Step 8: Results Persistence

**What Happens**

A comprehensive JSON record is appended to the JSONL output file:

```
{
  "scenario_id": 42,
  "doctor_model": "Mistral-7B-Instruct-v0.2",
  "ground_truth": "Pneumonia",
  "predicted_diagnosis": "Community-Acquired Pneumonia",
  "correct": true,
  "turn_count": 7,
  "differential_trajectory": [
    ["Pneumonia","Asthma","Bronchitis"],
    ["Pneumonia","TB","Bronchitis"],
    ...
  ],
  "tests_ordered": ["CBC", "Chest X-Ray"],
  "metrics": {
    "Diagnostic_Stability": 0.72,
    "Test_Rationality": 1.0,
    "Information_Efficiency": 1.28
  },
  "full_dialogue": [...]
}
```

If recency bias is enabled, the case summary is added to the `recent_case_summaries` FIFO queue (max $k = 5$) for injection into the next scenario's Doctor prompt.

### 9.2.9 Step 9: Final Summary

After all 107 scenarios complete, the system prints aggregate accuracy and saves results. If `-notify` is specified, a push notification is sent via `ntfy.sh`.

## 9.3 Complete Data Flow Diagram

`CLI args`

```
  |
  v
main() ----> Load Doctor Engine (NF4 + LoRA)
  |          Load Patient Engine (NF4, separate model)
  |          Load Moderator Engine (NF4)
  |          Load 107 scenarios from JSONL
  |
  v
FOR EACH SCENARIO:
  |
  +---> Create DoctorAgent (gets Objective only)
  |     Create PatientAgent (gets Symptoms only)
  |     Create MeasurementAgent (gets Tests only)
  |     Inject bias prompts if configured
  |
  +---> Build LangGraph DCG
  |       Nodes: reflection, speaker, patient, measurement
  |       Edges: deterministic + conditional (route_doctor)
  |       Compile graph
  |
  +---> app.invoke(initial_state)
  |       |
  |       +---> doctor_reflection (reasoning LoRA, T=0.1)
  |       |       Output: JSON DDx -> stored in trajectory
  |       |
  |       +---> doctor_speaker (dialogue LoRA, T=0.7)
  |       |       Output: spoken dialogue or REQUEST TEST or DIAGNOSIS READY
  |       |
  |       +---> route_doctor()
  |       |       DIAGNOSIS READY? -> END
  |       |       REQUEST TEST?    -> measurement_node -> back to reflection
  |       |       Otherwise?       -> patient_node -> back to reflection
  |       |
  |       +---> (repeat up to 10 turns)
  |       |
```

```
  |       +---> final_state returned
  |
  +---> compare_results() via Moderator LLM (semantic match)
  +---> ClinicalTrajectoryEvaluator.generate_full_report()
  +---> Append result to JSONL
  +---> Update recency bias queue
  |
  v
PRINT FINAL ACCURACY + SAVE
```

## 9.4 Potential Viva Questions — Code Workflow

**Likely Viva Questions**

1. **Q:** Walk me through what happens when you run an experiment.

   **A:** [Use the 9-step flow above: CLI → engine loading → dataset loading → scenario loop → graph construction → graph execution → evaluation → persistence → summary]

2. **Q:** What determines whether the doctor talks to the patient or orders a test?

   **A:** The `route_doctor()` function inspects the doctor's output string. If it contains "REQUEST TEST", routing goes to the measurement node. If "DIAGNOSIS READY" or turn limit reached, routing goes to END. Otherwise, routing goes to the patient node.

3. **Q:** What happens if the doctor fails to produce a diagnosis by the final turn?

   **A:** The system prompt is overridden with a forced diagnosis instruction. If the model still fails, a secondary `_force_final_diagnosis()` function is called with few-shot examples of the expected format. As a last resort, "Unable to determine" is emitted.

4. **Q:** How does the code enforce knowledge partitioning?

   **A:** Each agent class receives only its designated data slice during instantiation. `DoctorAgent` gets `examiner_information()`, `PatientAgent` gets `patient_information()`, `MeasurementAgent` gets `exam_information()`. The ground-truth diagnosis is only accessible to the `compare_results()` function via the Moderator.

5. **Q:** Where in the code does Lexical Leakage prevention happen?

   **A:** In `main()`, lines 1001–1010. If `patient_model_id != doctor_model_id`, a separate `LocalServerLLMWrapper` is instantiated. This creates two independent embedding manifolds ($\theta_{\text{Doctor}} \neq \theta_{\text{Patient}}$).

6. **Q:** How are the LoRA adapters swapped during execution?

   **A:** In `reflect_metacognition()` (line 717): `model.set_adapter("reasoning")` before DDx generation. In `inference_doctor()` (line 753): `model.set_adapter("dialogue")` before dialogue generation. The swap is a pointer operation that changes which $BA$ matrices are added to frozen $W_0$.

7. **Q:** What is ClinicalState and why is it a TypedDict?

   **A:** `ClinicalState` is the shared state schema passed between all LangGraph nodes. It's a `TypedDict` (not a regular dict) because LangGraph requires statically-typed state definitions for compile-time graph validation and type-safe node returns.